

1. How would you represent the input of the network (vector, matrix, multiple inputs etc.)
2. How would you design the output layer
3. Which activation functions would you use
4. Which Loss function
5. Which (possible) weight regularizers, initializers, etc.
6. On which hyperparameters would you perform the model selection and why
7. How to evaluate the test performance

Introduction

- Which kind of task
 - Supervised/unsupervised
 - Binary/multi-class
 - Single label/multi label
- What we have to predict
- *“Find best architecture that is the least complex possible”*
- *“find best hyperparameters that allows the model to perform good both on training and on test”*
- *“the model should be a trade off between computational complexity and accuracy”*
- *“avoid overfitting”*

How would you represent the input of the network (vector, matrix, multiple inputs etc.)

- Input are word sequences
- Need to find a way on how to create the embedding of a word
 - How to represent the word as a vector

Preprocessing

- Lowercase the text
- Split the text at white spaces
- Replace “-” with a white space so that we can split the word better
- Remove short messages (less than 3 words/characters)
- Remove punctuations
- Remove stopwords (NLTK list)
- Stemmatization
 - Snowball stemmer better than Porter Stemmatization
 - Return the stem of a word (*singing* → *sings*)
- Create a dictionary for unique word each

Embedding

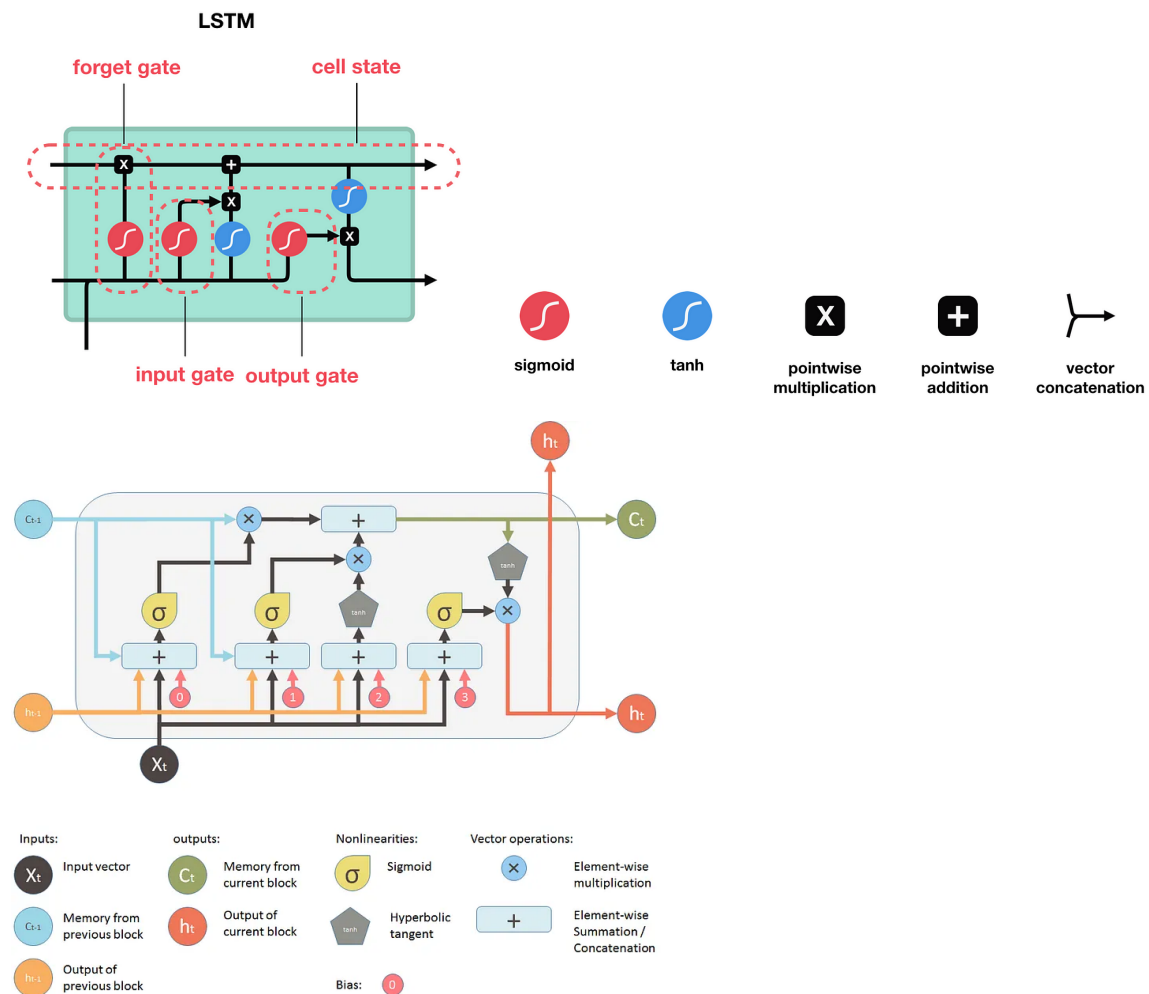
- How to represent the embedding of a word

- **One hot encoder**
 - Traditional way
 - Unique integer associated to each word
 - Most basic representation of any textual unit in NLP
 - Not very informative (*movies* and *movie* are independent) 😞
 - All words are equidistant 😞
 - Can not capture polysemy (*mouse* same for pc mouse and animal) 😞
- **Word2vec (CBOW)**
 - Unsupervised task (give a corpus without labels)
 - One of the most influential and widely used methods for word embedding
 - Take context word and try to predict a target word
 - Context words are first passed as an input to an embedding layer (initialized with some random weights)
 - The word embeddings are then passed to a lambda layer where we average out the word embeddings.
 - We then pass these embeddings to a dense SoftMax layer that predicts our target word. We match this with our target word and compute the loss and then we perform backpropagation with each epoch to update the embedding layer in the process.
- **GloVe**
 - Pre-trained word embeddings, based on global co-occurrence values
 - Fast to use, since they are pretrained, and capture global relations between words
 - The word embeddings are fed to an Embedding layer, after setting parameter “trainable” to False.
- **ELMo/BERT**
 - Contextualized embeddings: they capture long-term dependencies of the sentences, through the use of RNNs or Transformers
 - (Can be quite slow, so depending on size of input and computational power, one might prefer some more simple embeddings)

Create a dictionary and one hot encoding

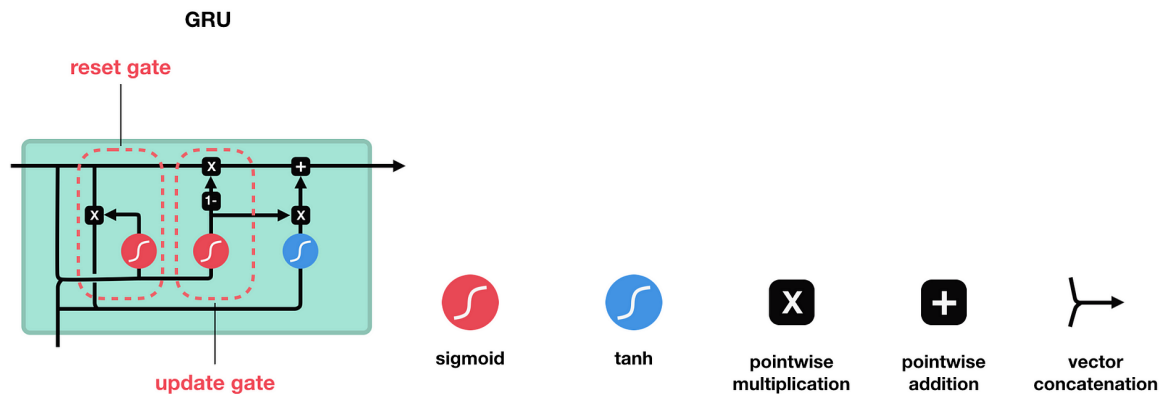
- Count how many unique words are in all the messages
- Create a dictionary with all the unique words where at each word is assigned a unique index (integer) - it could be in alphabetical order
- Create a one-hot encoder to represent each word as a binary vector of 0's and 1. Each vector is of the same length of the vocabulary and the element corresponding to the word's index is set to 1 and the rest to 0.

LSTM



- Hidden state (candidate for cell state update) + updated cell state to help determine new hidden state: **tanh** $\rightarrow$ (zero-centered, so easier to optimize, but suffer from vanishing gradient problem) + formula
 - Forget gate + input gate + output gate: **sigmoid** $\rightarrow$ (output normalized between 0 and 1 + suffer from vanishing gradient problem) + formula
 - Gate: decide which data is important and can be useful in the future and which data has to be erased
 - Forget gate**: which information is important and should be stored and which information we should forget. Takes two inputs: output generated by the previous cell + input of the current cell. The sigmoid activation function produce a value between 0 and 1 which decided which information to keep
 - Input gate**: add information to the neuron cell, it creates an array of information that has to be added.
 - Output gate**: select important information from the current cell and show it as output.

GRU



- Reset gate + update gate: sigmoid
- Candidate hidden state: tanh

How would you design the output layer

- Number of neurons in output layer = **number of classes**
- Softmax activation function if it's a multi-class classification

$$f(z) = \frac{e^z}{\sum_{i=1}^N e^{z_i}}$$

- Single output: value between 0 and 1
 - All the outputs: summed up are = 1
- Sigmoid activation function if it's a binary classification

$$\sigma(z) = \frac{1}{1+e^{-z}} + \text{graph}$$

Which activation functions would you use

Inside LSTM/GRU

- **Sigmoid**: (output normalized between 0 and 1 + suffer from vanishing gradient problem) $\sigma(z) = \frac{1}{1+e^{-z}} + \text{graph}$
- **Tanh**: (zero-centered, so easier to optimize, but suffer from vanishing gradient problem) $f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
- **Forget gate**: use sigmoid to decide what information we're going to throw away from the cell state. A 1 represents "completely keep this" while a 0 represents "completely get rid of this."

- **Input gate:** decide what new information we're going to store in the cell state. Decide which values we'll update (with a sigmoid). Next, a tanh layer creates a vector of new candidate values that could be added to the state. In the next step, we'll combine these two to create an update to the state.
- **Output gate:** push the values with tanh between -1 and 1

Which Loss function

- Different loss function, depends on how labels are represented and on the task we are dealing with:
 - Regression task with many outliers $\Rightarrow$ **mean absolute error**, since it is less sensitive to outliers (since it isn't squared, the outliers don't have too big weights)
 - Regression task $\Rightarrow$ **mean square error**. Compute the average square difference between predicted values and true values + formula
 - Binary classification $\Rightarrow$ **binary cross entropy loss**. Measure dissimilarity between predicted and true binary labels + formula
 - Multiclass with output as one-hot vector $\Rightarrow$ **categorical cross entropy loss**
 - Multiclass with output as integer $\Rightarrow$ **sparse categorical cross entropy loss**
 - **Square hinge loss** $\Rightarrow$ converge faster and better performance + more robust to noise in the training set labeling + quadratically increase with the error ($\rightarrow$ larger errors are punished more heavily)

Which (possible) weight regularizers, initializers, etc.

Regularizer

- Weight regularization
 - Add to the loss function of the model a cost associated with having large weights
 - Use to mitigate overfitting
 - Put constraint on the model
 - Force weight to take small values
 - Make distribution of weight more regular
- **L1:** cost added proportionally to the absolute value of the weight coefficients. Encourage the model to reduce the less important weights to zero + introduce sparsity in the weights by forcing more weights to be zero
- **L2:** cost added proportional to the square of the value of the weights coefficient. Penalize large weights + encourage weights to be small but do not drive them to exactly 0. More used for smaller deep learning models
- **Dropout:** randomly dropping out neurons
 - The output is set to 0
 - No weight update

- The idea is to introduce noise in the output values to break patterns
- Learn more robust and general representations

On which hyperparameters would you perform the model selection and why

- Learning rate: determine the step size at each iteration
 - How quickly or slowly the model learns from the training data
 - Common to start with a moderate learning rate in the range 0.1 to 0.001
- Batch size:
 - Number of sample processed before the model is updated
 - Divide the training dataset into smaller subsets
 - Each mini-batch is a subsample of training examples
 - Update the parameters based on the batch
- Epochs: number of times the entire dataset is passed in the model
- Number of layers
- Number of neurons
- Dropout rate: fraction of neurons randomly dropped during training to prevent overfitting
- Weight initialization
 - Random uniform or normal distribution
 - Xavier/Glorot initialization (Tanh/Sigmoid)
 - He initialization (ReLU)
- Optimizer: update the weights of the neural network during training
 - Stochastic gradient descent (SGD)
 - SGD with momentum
 - Adam
 - RMSprop
 - Adagrad

How to evaluate the test performance

- During the training part we can set a validation set that gives an estimate of model skill while tuning model's hyperparameters.
- Perform a manual search of the parameters that are difficult to tune in a grid search
- We apply a grid search to find some hyperparameters (like the activation function of the hidden layers). Perform on a small range of hyperparameters (very computationally expensive)
- Check the accuracy and f1_score on the train and test set
 - **Accuracy**: describe how the model performs across all classes. Useful when classes are equally important. Ratio between the number of correct predictions and the total number of predictions

- **F1_score**: popular metric for classification models that provides accurate results for both balanced and unbalanced dataset. Takes into account both precision and recall. Is two times the precision multiplied by the recall divided by the sum of precision and recall
- Plot a confusion matrix